1-G-Coin Problem

QUESTION:

Write a program to take value V and we want to make change for V Rs, and we have infinite supply of each of the denominations in Indian currency, i.e., we have infinite supply of { 1, 2, 5, 10, 20, 50, 100, 500, 1000} valued coins/notes, what is the minimum number of coins and/or notes needed to make the change.

Input Format:

Take an integer from stdin.

Output Format:

print the integer which is change of the number.

Example Input :

64

Output:

4

Explanaton:

We need a 50 Rs note and a 10 Rs note and two 2 rupee coins.

PROGRAM:

```
#include <stdio.h>
int main() {
  int V;
  int coins[] = {1000, 500, 100, 50, 20, 10, 5, 2, 1};
  int n = 9;
  int count = 0;
  scanf("%d", &V);
  for (int i = 0; i < n; i++) {
    count += V / coins[i];
    V = V % coins[i];
  }
```

```
    printf("%d", count);
    return 0;
}
```

OUTPUT:

	Input	Expected	Got	
✓	49	5	5	✓

Passed all tests!✓

2-G-Cookies Problem

QUESTION:

Assume you are an awesome parent and want to give your children some cookies. But, you should give each child at most one cookie.

Each child i has a greed factor g[i], which is the minimum size of a cookie that the child will be content with; and each cookie j has a size s[j]. If s[j] >= g[i], we can assign the cookie j to the child i, and the child i will be content. Your goal is to maximize the number of your content children and output the maximum number.

Example 1:

Input:

3

1 2 3

2

1 1

Output:

1

Explanation: You have 3 children and 2 cookies. The greed factors of 3 children are 1, 2, 3.

And even though you have 2 cookies, since their size is both 1, you could only make the child whose greed factor is 1 content.

You need to output 1.

Constraints:

1 <= g.length <= 3 * 10^4

0 <= s.length <= 3 * 10^4

1 <= g[i], s[j] <= 2^31 – 1

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
int compare(const void *a, const void *b)
{
   return (*(int *)a - *(int *)b);
}
int main()
{
   int n, m;
   scanf("%d", &n);
   int g[n];
   for (int i = 0; i < n; i++)
      scanf("%d", &g[i]);
   scanf("%d", &m);
   int s[m];
   for (int i = 0; i < m; i++)
      scanf("%d", &s[i]);
   qsort(g, n, sizeof(int), compare);
   qsort(s, m, sizeof(int), compare);
   int i = 0, j = 0, count = 0;
   while (i < n && j < m)
   {
```

```
        if (s[j] >= g[i])
        {
            count++;
            i++;
            j++;
        }
        else
        {
            j++;
        }
    }
    printf("%d", count);
    return 0;
}
```

OUTPUT:

	Input	Expected	Got	
✓	**2** **1 2** **3** **1 2 3**	**2**	**2**	✓

4-G-Array Sum max problem

QUESTION:

Given an array of N integer, we have to maximize the sum of arr[i] * i, where i is the index of the element (i = 0, 1, 2, ..., N).Write an algorithm based on Greedy technique with a Complexity O(nlogn).

Input Format:

First line specifies the number of elements-n

The next n lines contain the array elements.

Output Format:

Maximum Array Sum to be printed.

Sample Input:

5

2 5 3 4 0

Sample output:

40

Program:

```
#include <stdio.h>

#include <stdlib.h>

int compare(const void *a, const void *b)

{

  return (*(int *)a - *(int *)b);

}

int main()

{

  int n;

  scanf("%d", &n);

  int arr[n];
```

```
    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }
    qsort(arr, n, sizeof(int), compare);
    long long sum = 0;
    for (int i = 0; i < n; i++)
    {
        sum += (long long)arr[i] * i;
    }
    printf("%lld\n", sum);
    return 0;
}
```

OUTPUT:

	Input	Expected	Got	
✓	5 2 5 3 4 0	40	40	✓

	Input	Expected	Got	
✓	10 2 2 2 4 4 3 3 5 5 5	191	191	✓
✓	2 45 3	45	45	✓

Passed all tests! ✓

5-G-Product of Array elements-Minimum

QUESTION:
Given two arrays array_One[] and array_Two[] of same size N. We need to first rearrange the arrays such that the sum of the product of pairs(1 element from each) is minimum. That is SUM (A[i] * B[i]) for all i is minimum.

For example:

Input	Result
3 1 2 3 4 5 6	28

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
int ascending(const void *a, const void *b)
{
   return (*(int *)a - *(int *)b);
}
int descending(const void *a, const void *b)
{
   return (*(int *)b - *(int *)a);
}
int main()
{
   int n;
   scanf("%d", &n);
   int array_One[n], array_Two[n];
   for (int i = 0; i < n; i++)
   {
```

```
        scanf("%d", &array_One[i]);
    }
    for (int i = 0; i < n; i++)
    {
        scanf("%d", &array_Two[i]);
    }
    qsort(array_One, n, sizeof(int), ascending);
    qsort(array_Two, n, sizeof(int), descending);
    long long sum = 0;
    for (int i = 0; i < n; i++)
    {
        sum += (long long)array_One[i] * array_Two[i];
    }
    printf("%lld\n", sum);
    return 0;
}
```

OUTPUT:

	Input	Expected	Got	
✓	3 1 2 3 4 5 6	28	28	✓
✓	4 7 5 1 2 1 3 4 1	22	22	✓
	5 20 10 30 10 40 8 9	590	590	

	Input	Expected	Got	
	4 3 10			

Passed all tests! ✓